

For Starters

If you have ample time to study:

- Be comfortable at converting from/to Hexadecimal, binary and decimal
- MASTER cache translations

THIS REVIEW DOES NOT COVER THE HOMEWORKS

Definitions

As a warm up

Different pointers or expressions
access the same location in
memory. What is this called?

Memory Aliasing

Nonvolatile memory designed to hold fixed information, such as firmware, that is not normally rewritten during ordinary operation.

Read Only Memory (ROM)

A signal that causes the processor to temporarily suspend normal execution and transfer control to a handler.

Interrupt

Removing a block from the
cache, usually to make room for
another block.

Eviction

Memory that loses its stored contents when power is removed.

Volatile Memory

A communication pathway used to connect peripheral devices to the rest of the computer system.

I/O Bus

A communication pathway that carries data, addresses, and control signals between the memory controller and main memory.

Memory Bus

A mechanism that transfers data between an I/O device and main memory without requiring the CPU to copy each individual piece of data.

Direct Memory Access (DMA)

A programmer-specified condition that is checked to catch programming errors. If the check fails, the program typically reports the failure and terminates.

Assertion

A processor technique that guesses a branch's outcome so execution can proceed before the actual outcome is known.

Branch Prediction

An optimization that moves a calculation to a place where it executes fewer times, such as outside a loop, while preserving behavior.

Code Motion

A processor design that can issue multiple instructions during a single clock cycle.

Superscalar

The time required for one operation to complete, measured from its start until its result is available.

Latency

The amount of work completed
per unit of time, such as
instructions completed per
second.

Throughput

A cache write policy that updates the cached copy and marks it dirty, postponing the update to the next memory level until eviction or a flush.

Write-Back

A cache write policy that sends every write to a cached block through to the next memory level as well.

Write-Through

Code that can execute correctly at different loading addresses without modifying the code itself to account for its location.

Position Independent Code (PIC)

Matching a reference to a
function or global variable with
the definition that provides it.

Symbol Resolution

Adjusting address references in
code and data to match the
locations assigned to program
components.

Relocation

Caches

Locality is why caches work

Principle of Locality: Programs tend to reuse recently accessed locations and access locations near one another.

Temporal Locality: Something accessed recently will be reused again. EX:
Reusing the same memory address again and again

Spatial Locality: Locations near something just accessed will be accessed soon.
EX: if `a[0]` gets accessed, `a[1]` will probably follow

Caches load entire blocks

Suppose:

An integer is 4 bytes

Each cache block holds 64 bytes

`a[0]` is the start of a cache block

The cache starts empty

HIT OR MISS?

`a[0]`? = MISS (Cache not loaded yet)

What about `a[8]`? = HIT

`a[16]`? = MISS

One block holds:

$64 \text{ bytes} / 4 \text{ bytes per integer} = 16 \text{ integers}$

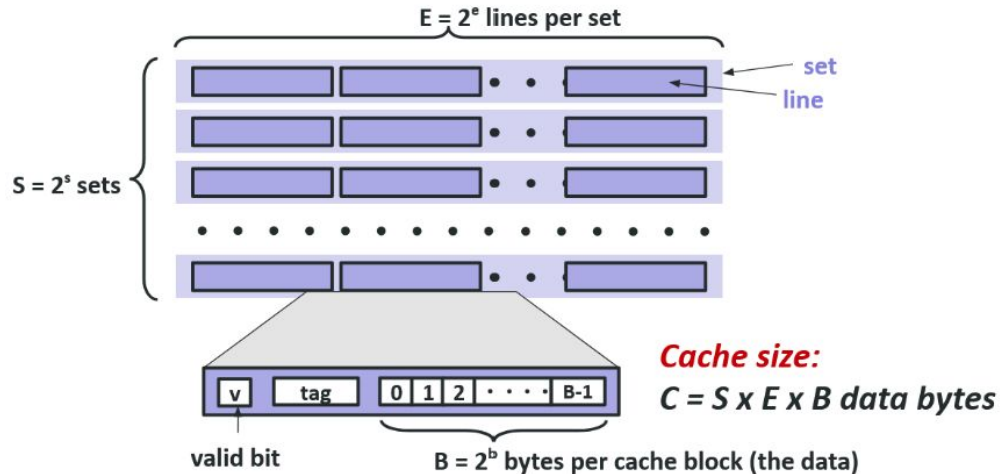
What will reading `a[0]` do?

It will load in the entire cache block containing `a[0]` through `a[15]` (included).

Cache organization

A cache is organized into sets, lines, and blocks. Visualizing this can be important for cache design tradeoffs!

General Cache Organization (S, E, B)



EX: A cache with 4 sets, 2 lines per set, and 8 byte blocks holds:

$4 \times 2 \times 8 = 64$ bytes in the cache

Cache Size

A cache is used in a system with 32-bit addresses. It contains 8 sets, 4 lines per set, and 32-byte blocks.

$$C = S * E * B$$

$$C = 8 \times 4 \times 32 = 1024 \text{ bytes}$$

Cache misses

Compulsory/Cold: First access to a block.

Conflict: Blocks competing for the same set (fix: Higher associativity can reduce these misses)

Capacity: The cache is too small to retain the needed working set, even without set-placement restrictions.

Cache Architectures

Architecture	Where can a memory block go?	Organization	Design tradeoffs
Direct Mapped	Exactly one specific line	One line per set: $E = 1$	Simple, inexpensive hardware and fast lookup: only one tag to check. More prone to conflict misses.
E-way set associative	Any of the E lines in one specific set	Multiple lines per set	Reduces conflict misses by allowing multiple blocks per set. Increases hardware complexity and potentially lookup time.
Fully associative	Any line in the entire cache	One set: $S = 1$	Eliminates conflict misses: a block can go anywhere. Must check tags across all cache lines and choose a replacement. Expensive and complex.

More Design Trade Offs

Change	Hold fixed	Structural effect	Performance trade-off
Higher associativity	Capacity, block size	Fewer sets; more lines per set	Usually fewer conflict misses, but more tag comparisons, replacement complexity, and potentially longer hit time.
Lower associativity	Capacity, block size	More sets; fewer lines per set	Simpler, potentially faster lookup, but more conflict misses.
Larger blocks	Capacity, associativity	Fewer blocks and sets	Can exploit spatial locality more, but unused bytes waste space, fewer distinct blocks fit, and miss penalty generally increases.
Smaller blocks	Capacity, associativity	More blocks and sets	Less wasted data and shorter transfers, but more misses during sequential access and more tag entries to store.
More sets	Associativity, block size	Larger cache capacity	Usually fewer capacity/conflict misses, but more hardware area, power, and potentially longer hit time.
Fewer sets	Associativity, block size	Smaller cache capacity	Smaller, potentially faster hardware, but fewer blocks fit and misses can increase.

Cache addresses

An address tells the cache where to look and what to check

A cache address, divides the address bits into three parts:

Tag: Is this the right memory block?

Set index: Which set should I search?

Block offset: Which byte inside the block do I want

t s b.

$$t = m - s - b. \quad s = \log_2 (S). \quad b = \log_2 (B)$$

m is the total number of address bits

Run through:

- Use the set index to select one set
- Check the lines in that set for a matching tag
- A match counts only if the lines valid bit is 1
- On a hit, use the offset to select the requested byte or starting position

How to do Cache lookup questions

1. Convert hexadecimal to binary
2. Split the bits into tag, set, block offset (t s b)
3. Traverse the cache, finding the indicated set
4. Check for valid bit and matching tag
5. If found, its a hit. Otherwise its a miss

Cache Demo

Block Number:	Set	Valid	Tag
0	0	0	2
1	1	1	1
2	2	1	3
3	3	1	1

2 bits of set, 2 bits of block offset, 2 bits of tag

What cache block does 0x1D refer to?

0x1D = 011101

t s b = 01 11 01

Convert from binary to decimal

01 = 1, 11 = 3, 01 = 1

Now look at your cache table

Set 3, Block number 3, Valid bit 1,
Tag 1. CACHE HIT.

Cache Demo

Block Number:	Set	Valid	Tag
0	0	0	7
1	0	1	3
2	1	1	3
3	1	0	2
4	2	1	1
5	2	1	6
6	3	0	5
7	3	1	1

2 bits of set, 2 bits of block offset, 3 bits of tag

What cache block does 0x69 refer to?

0x69 = 01101001

We only need 7 bits. So remove the leading 0. = 1101001

t s b = 110 10 01

Binary to decimal: 6, 2, 1

Look at rows belonging to set 2. Both have valid bit but only one has a tag of 6. CACHE HIT.

Cache hit miss rate questions

First, must calculate elements per block.

Elements per block = Block size/Element size

EX:

Block size 16 bytes, element size 8 byte doubles, 2 elements per block

Block size 64 bytes, element size 4 byte integer, 16 elements per block

The fraction of misses depends on these numbers.

Assumes sequential access to aligned, initially uncached blocks, with each block retained until its elements have been accessed.

Elements per block	Access pattern within each new block	Miss rate
2	1 miss (compulsory), then 1 hit	1/2
4	1 miss (compulsory), then 3 hits	1/4
8	1 miss (compulsory), then 7 hits	1/8
16	1 miss, then 15 hits	1/16

Typically, arrays follow the form `Arr[row][column]`

To move across a row: `A[2][0]`, `A[2][1]`, `A[2][2]`

Traversing a column: `A[0][2]`, `A[1][2]`, `A[2][2]`

Traversing a row moves through consecutive elements while traversing through a column jumps through entire rows

Row is first, column is second

You want the first bracket to be fixed, and the second to change by 1 (if the second is the inner most loop)

For triple nested loops, focus on the innermost loop first

```
1 for (i = 0; i < n; i++) {  
2     for (j = 0; j < n; j++) {  
3         double sum = 0;  
4  
5         for (k = 0; k < n; k++) {  
6             sum += A[i][k] * B[k][j];  
7         }  
8  
9         C[i][j] = sum;  
10    }  
11 }
```

In the innermost loop, k will change while i and j stay fixed.

Inspected each array access,

A[i][k] changes k in the second bracket

B[k][j] changes k in the first bracket

C[i][j] does not change k. Written after the loop finishes.

There are two array reads in this example.
So miss rate =
 $1.125/2 = 56.25\%$

Assume the problem gives 64 byte blocks and 8 byte doubles

A: misses $\frac{1}{8}$ per inner iteration. B misses 1 time per inner iteration.

Misses per inner iteration = $\frac{1}{8} + 1 = \frac{9}{8}$

Loop Ordering notice

Loop ordering changes which loop is outermost and which is innermost. The goal is to access data in an order that uses the cache effectively.

For C arrays, neighboring elements within a row are adjacent in memory. Making the second index change fastest therefore improves spatial locality.

Even with sequential access, a large matrix may not fit in the cache. By the time the program needs an element again, other accesses may have evicted it. Good spatial locality does not guarantee that data survives until its next use.

Blocking

Blocking, also called tiling, divides the computation into smaller matrix regions. The program repeatedly uses those regions while they can remain in the cache.

Within each small matrix multiplication, an element of A contributes to several output elements, and an element of B is used for several output rows. Those repeated uses happen close together.

Temporal locality improves because the same elements are reused before eviction. Spatial locality is exploited by traversing neighboring elements within tile rows.

Average Access Time

Average memory access time, or AMAT, measures the average time needed to satisfy a memory request.

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

$$\text{Miss rate} = 1 - \text{Hit rate}$$

Hit time is the time spent checking the cache and accessing its data. In this model, you spend that lookup time even when the cache misses.

Miss penalty is the additional time needed after a miss. It does not include the lookup time already counted. computationstructures.org

For example, a cache with a 90% hit rate has a 10% miss rate:

$$\text{Miss rate} = 1 - 0.90 = 0.10$$

Extending the Formula to Multiple Levels

With multiple caches, the miss penalty includes the average time needed to access the remaining levels.

Let T_1 , T_2 , and T_3 represent each cache's lookup time. Let M_1 , M_2 , and M_3 represent their miss rates. T_{memory} is the main-memory access time.

One cache, then main memory:

$$\text{AMAT} = T_1 + M_1 \times T_{\text{memory}}$$

Two caches, then main memory:

$$\text{AMAT} = T_1 + M_1 \times (T_2 + M_2 \times T_{\text{memory}})$$

Three caches, then main memory:

$$\text{AMAT} = T_1 + M_1 \times [T_2 + M_2 \times (T_3 + M_3 \times T_{\text{memory}})]$$

Example

Find the average memory access time using the following information.

Level	Local hit rate	Time spent accessing this level
L1 cache	90%	1 ns
L2 cache	80%	5 ns
L3 cache	50%	20 ns
Main memory	Serves all remaining requests	100 ns

Example Solution

First, convert hit rates into miss rates: $M1 = 1 - 0.90 = 0.10$ $M2 = 1 - 0.80 = 0.20$ $M3 = 1 - 0.50 = 0.50$

Then, work from the bottom up.

Start at L3.

Every request reaching L3 spends 20 ns checking it. Half then require another 100 ns for main memory. Average time starting at L3
 $= T3 + M3 \times T_{\text{memory}} = 20 + 0.50 \times 100 = 70 \text{ ns}$

Move up to L2.

Every request reaching L2 spends 5 ns checking it. The 20% that miss continue to L3, where the average remaining time is 70 ns.

Average time starting at L2 $= T2 + M2 \times \text{Average time starting at L3} = 5 + 0.20 \times 70 = 19 \text{ ns}$

Finally, calculate the average starting at L1.

Every request spends 1 ns checking L1. The 10% that miss continue to L2, where the average remaining time is 19 ns.

$AMAT = T1 + M1 \times \text{Average time starting at L2} = 1 + 0.10 \times 19 = 2.9 \text{ ns}$

$AMAT = 1 + 0.10 \times [5 + 0.20 \times (20 + 0.50 \times 100)] = 2.9 \text{ ns}$

Optimization

Optimizations

Compiler optimization changes how a program executes so it can perform the same work more efficiently.

Importantly, The optimized program must preserve the required behavior of the original program.

Choosing a better algorithm or data structure is still up to the programmer

What the compiler can't optimize

The compiler will not make a change if it is unsafe.

Obstacle	Why it matters
Memory aliasing	Writing through one pointer might change a value accessed through another.
Function side effects	Removing or moving a call might change global variables, memory, or output.
Data dependencies	One calculation may require the result of an earlier calculation.
Floating-point rounding	Changing the grouping of calculations can change the answer.

Pointers

```
int x = 10;
```

```
int *p = &x;
```

Expression

Meaning

x

The value stored in x

&x

The address of x

p

The address stored in the pointer

*p

The value at that address

*p = 20

Changes x to 20

p++

Moves the pointer to the next element

(*p)++

Increases the pointed-to value by 1

Track the program

```
1  int a[] = {3, 5, 7};
2  int *p = a;
3  int *q = p;
4
5  p++;
6  *q += *p;
7  (*p)++;
```

Step	p points to	q points to	Array
Initial setup	a[0]	a[0]	{3, 5, 7}
p++	a[1]	a[0]	{3, 5, 7}
*q += *p	a[1]	a[0]	{8, 5, 7}
(*p)++	a[1]	a[0]	{8, 6, 7}

Memory Aliasing

Aliasing occurs when different references access the same memory location

```
1  int x = 10;  
2  int *p = &x;  
3  int *q = &x;  
4  
5  *q = 25;  
6  printf("%d\n", *p);  
7
```

What is the value of dereferenced p after the program runs?

25

Local Accumulators reduce repeated memory writes

```
1  *out = 0;  
2  
3  for (int i = 0; i < n; i++)  
4      *out += a[i];  
5
```

```
1  int total = 0;  
2  
3  for (int i = 0; i < n; i++)  
4      total += a[i];  
5  
6  *out = total;  
7  
8
```

A memory write on every loop iteration vs 1 at the very end

Code motion

Code motion moves a computation outside a loop when its result stays unchanged and moving it preserves behavior.

```
1  for (long j = 0; j < n; j++)  
2      a[n*i + j] = b[j];  
~
```

```
1  long base = n*i;  
2  
3  for (long j = 0; j < n; j++)  
4      a[base + j] = b[j];
```

Reduction in Strength

Replaces an operation with a cheaper, equivalent operation

Examples include replacing multiplication with addition

Or using bit shifting to replace multiplication or division

Shifts can replace suitable multiplication or division by powers of two when the replacement preserves the original behavior.

```
1  int y = 10;  
2  int x = y * 3;  
3
```

Would replacing $y * 3$ with $y \ll 3$ be correct?

Actually understand how shifting works.
Shifting left by 3 means doing $2^3 = 8$.
Making the answer become $10 * 8 = 80$.
Not 30.

Dead code elimination

```
1 v int f(void) {  
2     int x = 3;  
3     x = 5;  
4     return x;  
5 }  
6
```

```
1 v int f(void) {  
2     return 5;  
3 }  
4
```

The initial value 3 is never used because it is overwritten by 5 before the return.

Loop unrolling

```
1 unsigned total = 0;
2 int i = 0;
3
4 for (; i + 1 < n; i += 2) {
5     total += a[i];
6     total += a[i + 1];
7 }
8
9 for (; i < n; i++)
10     total += a[i];
```

Processes more elements per loop iteration. The total work for the loop ends up being the same, but the loop overhead is reduced.

Loop unrolling

```
1 unsigned total = 0;
2 int i = 0;
3
4 for (; i + 1 < n; i += 2) {
5     total += a[i];
6     total += a[i + 1];
7 }
8
9 for (; i < n; i++)
10     total += a[i];
```

In this example, both operate under the same accumulator “total.” The second addition needs the result of the first, creating a dependency chain. A processor is capable of doing multiple calculations at once, but dependency forces the operation to wait. The solution is to have separate accumulators.

```

01 int calls = 0;
02
03 int next_value(void)
04 {
05     return ++calls;
06 }
07
08 int analyze(int *a, int *out)
09 {
10     *out = 0;
11     for (int i = 0; i < 3; i++)
12         *out += a[i];
13
14     int extra = 0;
15     for (int j = 0; j < 3; j++)
16         extra += next_value();
17
18     return *out + extra;
19 }
20
21 int main(void)
22 {
23     int a[] = {1, 2, 3};
24     int *p = a;
25     int *q = p + 1;
26     int result = analyze(p, q);
27     printf("%d %d %d\n", result, a[1], calls);
28 }

```

Suppose lines 10-12 get replaced with:

```

1  int total = 0;
2  for (int i = 0; i < 3; i++)
3      total += a[i];
4
5  *out = total;
6

```

Suppose lines 14-16 get replaced with:

```

1  int extra = 0;
2  int value = next_value();
3
4  for (int j = 0; j < 3; j++)
5      extra += value;
6
7

```

```

01 int calls = 0;
02
03 int next_value(void)
04 {
05     return ++calls;
06 }
07
08 int analyze(int *a, int *out)
09 {
10     *out = 0;
11     for (int i = 0; i < 3; i++)
12         *out += a[i];
13
14     int extra = 0;
15     for (int j = 0; j < 3; j++)
16         extra += next_value();
17
18     return *out + extra;
19 }
20
21 int main(void)
22 {
23     int a[] = {1, 2, 3};
24     int *p = a;
25     int *q = p + 1;
26     int result = analyze(p, q);
27     printf("%d %d %d\n", result, a[1], calls);
28 }

```

```

1 int total = 0;
2 for (int i = 0; i < 3; i++)
3     total += a[i];
4
5 *out = total;
6

```

out points to a[1], so writing to *out also changes the array being read.

Originally, *out = 0 sets a[1] to zero. The loop then updates it to 1 → 2 → 5.

The replacement adds the original values: 1 + 2 + 3 = 6, then writes 6 into a[1].

New output: 12 6 3

Memory aliasing makes delaying the writes change the result.

```

01 int calls = 0;
02
03 int next_value(void)
04 {
05     return ++calls;
06 }
07
08 int analyze(int *a, int *out)
09 {
10     *out = 0;
11     for (int i = 0; i < 3; i++)
12         *out += a[i];
13
14     int extra = 0;
15     for (int j = 0; j < 3; j++)
16         extra += next_value();
17
18     return *out + extra;
19 }
20
21 int main(void)
22 {
23     int a[] = {1, 2, 3};
24     int *p = a;
25     int *q = p + 1;
26     int result = analyze(p, q);
27     printf("%d %d %d\n", result, a[1], calls);
28 }

```

```

1  int extra = 0;
2  int value = next_value();
3
4  for (int j = 0; j < 3; j++)
5      extra += value;
6
7

```

Originally, next_value() runs three times, returning 1, 2, 3. Therefore, extra = 6 and calls = 3.

The replacement calls it once, then adds the returned value three times: $1 + 1 + 1 = 3$. Therefore, extra = 3 and calls = 1.

New output: 8 5 1

Moving the call changes both its side effects and the values added.

Concept Questions

“4 concept questions about: pointers, memory aliasing, locality, and compiler optimizations/toolchain.”

Pointers

What is the difference between `p`, `*p`, and `&p` when `p` is a pointer?

Answer: `p` is the address stored in the pointer. `*p` accesses the object at that address. `&p` is the address of the pointer variable itself. The pointer and the object it points to occupy separate memory locations.

Pointers

How does `p++` differ from `(*p)++`? Does incrementing a pointer always move it forward by one byte?

Answer: `p++` moves the pointer to the next element of its pointed-to type. For example, if an `int` occupies four bytes, incrementing an `int *` advances the address by four bytes. `(*p)++` leaves the pointer's address unchanged and increases the pointed-to value by one.

Pointers

When a function receives a pointer argument, can it modify the caller's data? Can assigning a new address to that parameter change the caller's pointer?

Answer: The function receives a copy of the pointer's address. It can modify the caller's data by dereferencing that address. However, assigning a different address to the parameter only changes the function's local pointer copy. Changing the caller's pointer itself requires access to that pointer variable, commonly through a pointer to a pointer.

Pointers

Does a pointer being non-NULL guarantee that it is safe to dereference?

Answer: No. A non-NULL pointer might be uninitialized, point outside a valid object, or point to memory whose lifetime has ended. For example, freeing allocated memory does not automatically set every pointer to that memory to NULL. Such pointers become dangling pointers, and dereferencing them is invalid.

Memory Aliasing

What is memory aliasing? Does copying one pointer into another create a separate copy of the pointed-to data?

Answer: Memory aliasing occurs when different references access the same memory location. Copying a pointer copies its address, not the data at that address. Both pointers can therefore access the same object, and a change made through one can be observed through the other.

Memory Aliasing

A program reads through `p`, writes through `q`, and then reads through `p` again.
Why might the compiler need to repeat the read?

Answer: `p` and `q` might point to the same object. The write through `q` could change the value accessed through `p`, making the previously loaded value outdated.
Reusing the earlier value is safe only if the compiler can establish that doing so preserves the program's behavior.

Memory Aliasing

Why can replacing repeated writes through an output pointer with a local accumulator change a program's result?

Answer: The output pointer might refer to an element of the input array. In the original program, each write could change a value that a later iteration reads. A local accumulator delays those writes until the end, so later iterations might read different values. The optimization is valid only when changing the timing of the writes preserves behavior.

Memory Aliasing

If an input pointer is declared `const int *a`, can the compiler assume that the values in the array never change?

Answer: No. This declaration prevents modification through `a`, but another pointer may still modify the same underlying data if that data is not itself defined as `const`. For example, an output pointer could point into the array. A pointer-to-`const` declaration does not establish that the input and output occupy separate memory.

Locality

What is the difference between spatial locality and temporal locality?

Answer: Spatial locality means accessing locations near one another, such as consecutive array elements. Temporal locality means accessing the same location again soon, such as repeatedly looking up the same table entry. Caches exploit spatial locality by loading entire blocks and temporal locality by retaining recently accessed data.

Locality

Why does sequential array traversal usually have better spatial locality than linked-list traversal? Does using an array automatically guarantee good locality?

Answer: Array elements occupy consecutive memory locations, so a loaded cache block often contains several elements that will be accessed next. Linked-list nodes may be scattered throughout memory. However, an array can still have poor locality if the program accesses widely separated elements or uses a large stride that skips most of each loaded block.

Locality

For a C two-dimensional array, why is it usually better for the innermost loop to change the second index? Does the loop variable's name matter?

Answer: C stores two-dimensional arrays in row-major order, so consecutive elements within a row are adjacent in memory. Keeping the first index fixed while increasing the second index by one accesses those adjacent elements. The variable's name does not matter; what matters is which index changes fastest.

Locality

What is cache blocking, and why can it improve performance even when a program performs the same calculations?

Answer: Cache blocking divides a large computation into smaller regions of data that can fit in the cache. The program performs several operations using one region before moving on, increasing the chance that needed data is still cached. The arithmetic can remain the same while fewer values must be fetched repeatedly from slower memory. Blocks that are too large may lose this benefit.

Compiler Optimizations

Is an unchanged result enough to make moving a calculation outside a loop safe?

Answer: No. The compiler must also consider whether the calculation originally executes at all and whether moving it changes observable effects. A loop might execute zero times, while a calculation placed before it would execute regardless. Safe code motion must preserve behavior as well as the calculated value.

Compiler Optimizations

Function A performs one addition and is called millions of times inside a loop. Function B performs a lengthy calculation and is called once. Which is generally the stronger candidate for inlining, and why?

Answer: Function A. Call and return overhead can account for a substantial fraction of the work of a tiny function, especially across millions of calls. Inlining can remove that repeated overhead and expose its calculation to surrounding optimizations. For Function B, the one-time call overhead is likely small compared with the calculation itself.

Compiler Optimizations

If a function's return value is unused, can the compiler automatically remove the function call?

Answer: No. The function might produce output, modify memory, change a global variable, or perform another observable action. Removing the call would remove those effects. The compiler may eliminate the call only when it can establish that doing so preserves the program's required behavior. An unused return value alone is insufficient.

Compiler Optimizations

Does loop unrolling always make a program faster? What limitations can remain after a loop is unrolled?

Answer: Unrolling reduces loop-control overhead by processing multiple elements per iteration, but dependencies may remain if every operation updates the same accumulator. It also increases code size and may increase register pressure. Multiple accumulators can expose more independent work, but performance still depends on the processor and workload. Any leftover elements must also be handled correctly.

Physical Memory

(no HDD components but they will be on the exam)

Memory Hierarchy

From fastest access to slowest access:

Registers → L1 cache → L2 cache → L3 cache → Main memory (DRAM) → SSD → HDD

Moving down the hierarchy generally means greater capacity, lower cost per byte, and higher access latency. Moving up means smaller, faster, more expensive storage.

Caches typically use SRAM, which is fast but relatively expensive. Main memory uses DRAM, which is denser and cheaper but requires periodic refreshing.

Registers, caches, and ordinary RAM are volatile. SSDs and HDDs are nonvolatile, so their stored data survives power loss.

Memory Hierarchy Tradeoffs

Moving from HDD → SSD → main memory (DRAM) → cache (SRAM), access generally becomes faster, cost per byte increases, and typical storage capacity decreases.

Moving down the simplified memory hierarchy generally means slower access, lower cost per byte, and greater capacity. Lower levels also have greater storage density. (on average). Faster memory prioritizes quick access, while lower levels prioritize storing more data.

HDD Access Time

An HDD stores data on rotating magnetic platters. Each surface contains circular tracks, which are divided into sectors.

Seek time: The read/write head moves to the track containing the requested sector.

Rotational latency: Once the head reaches the track, it waits for the requested sector to rotate underneath it.

Transfer time: Once the sector reaches the head, the drive reads its data.

Average access time = Average seek time + Average rotational latency + Transfer time

6,000 RPM, 6 ms average seek time, 500 sectors per track.

1. Full rotation time

Full rotation time (ms) = $60,000 \div \text{RPM} = 60,000 \div 6,000 = \mathbf{10 \text{ ms}}$

Use 60,000 because there are 60,000 milliseconds in one minute.

2. Average rotational latency

Average rotational latency = Full rotation time $\div 2 = 10 \div 2 = \mathbf{5 \text{ ms}}$

Shortcut: Average rotational latency (ms) = $30,000 \div \text{RPM}$

3. Transfer time

Transfer time = (Sectors requested $\div$ Sectors per track) $\times$ Full rotation time = $(1 \div 500) \times 10 = \mathbf{0.02 \text{ ms}}$

For one sector, this simplifies to: Full rotation time $\div$ Sectors per track.

4. Average access time

Average access time = Average seek time + Average rotational latency + Transfer time = $6 + 5 + 0.02 = \mathbf{11.02 \text{ ms}}$

The average seek time is given in the question; it cannot be calculated from RPM.

Feature	SSD	HDD
Storage technology	Electronic flash memory	Magnetic rotating platters
Moving parts	None	Spinning platters and moving heads
Mechanical access delays	No seek or rotational delay	Seek time and rotational latency
Random-access performance	Generally much faster	Slower because of mechanical positioning
Cost per byte	Generally higher	Generally lower
Physical shock	More resistant	More vulnerable to mechanical damage
Wear concerns	Limited flash program/erase endurance	Mechanical wear and other failures
Retains data without power?	Yes	Yes